Real-Time Chat Application (WebSocket) Interview Questions

1. Project Overview

• Tell me about your project.

• What problem does it solve?

• Why did you choose WebSockets?

• Explain your project architecture.

• What was your role?

2. Architecture

• Flow: React Client → WebSocket → Node.js + Express + Socket.IO → MongoDB → Receiver.

• Explain the request and response flow.

3. WebSocket

• What is WebSocket?

• Why WebSocket instead of HTTP?

• Difference between HTTP and WebSocket.

• What is a persistent connection?

• What is Full Duplex communication?

• What is the WebSocket handshake?

• Why does WebSocket start with HTTP?

• Difference between ws:// and wss://

• What is Socket.IO?

• Why Socket.IO instead of raw WebSocket?

• Does Socket.IO always use WebSocket?

4. Connection Flow

• Client opens app → HTTP Handshake → Upgrade Request → WebSocket Connection → Connection Established → Bidirectional Messaging

5. Socket.IO

• What are emit() and on()?

• What is broadcast?

• What are rooms?

• What are namespaces?

• What is socket.id?

• Difference between io.emit() and socket.emit().

• How do users join and leave rooms?

6. Message Flow

• Sender types → socket.emit() → Server receives → Store in MongoDB → Server emits → Receiver receives instantly.

• What if receiver is offline?

7. MongoDB

• Why MongoDB?

• Message schema fields: senderId, receiverId, message, timestamp.

• Why ObjectId?

• MongoDB vs SQL.

8. Authentication

• How do users log in?

• JWT authentication.

• Where is JWT stored?

• How do you authenticate socket connections?

9. Online/Offline

• How do you track online users?

• How do you detect disconnects?

• How do you show online status?

10. Events

• connection

• disconnect

• joinRoom

• leaveRoom

• sendMessage

• receiveMessage

• typing

• stopTyping

11. Typing Indicator

• How does typing work?

• Why isn't typing status stored in the database?

12. Read Receipts

• How do you implement Seen status?

• What happens if the receiver is offline?

13. Scalability

• Can it support 1 million users?

• How would you scale it?

• Redis adapter

• Load balancer

• Horizontal scaling

14. Database

• What indexes would you create?

• How do you fetch chat history?

15. REST API vs WebSocket

• REST: Login, Register, Fetch History, Profile.

• WebSocket: Real-time Messaging, Typing, Online Status, Read Receipts.

16. Security

• Why JWT?

• Why wss://?

• How do you validate messages?

• How do you prevent spam?

17. Performance

• Why WebSocket is faster than polling?

• What is latency?

• How do you optimize performance?

18. Error Handling

• What if MongoDB is down?

• How do you reconnect sockets?

• How do you handle network failures?

19. Deployment

• Deploying Socket.IO.

• WebSockets behind Nginx.

• HTTPS and WSS.

20. Coding Questions

• Implement a WebSocket server.

• Socket.IO connection.

• Join/Leave Room.

• Broadcast messages.

• Authentication middleware.

21. Resume Questions

• Biggest challenge?

• Favorite feature?

• What would you improve?

• Most difficult bug?

Must-Know Topics

• WebSocket lifecycle

• HTTP vs WebSocket

• Socket.IO events

- JWT Authentication
- MongoDB indexing
- Real-time message flow
- Online/offline tracking
- Typing indicators
- Reconnection
- Scaling with Redis